

Automation with Selenium

Assignment 2: Synchronization & Explicit Waits

1. Problem Statement: -

Navigate to a page with dynamic content. Do not use `time.sleep()`. Implement `WebDriverWait` along with `expected_conditions` to wait until the text element is visible before extracting it.

2. Objective: -

To understand and apply explicit synchronization (`WebDriverWait`) for handling dynamically loaded web elements.

3. Tools, Software, and Concepts Used: -

Python 3.x, Selenium WebDriver, `WebDriverWait`, `Expected Conditions`.

4. Source Code / Implementation: -

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
import time

driver = webdriver.Chrome()
driver.get("https://the-internet.herokuapp.com/dynamic_loading/1")
time.sleep(2)
driver.find_element(By.CSS_SELECTOR, "#start button").click()
wait = WebDriverWait(driver, 10)
hello_text = wait.until(EC.visibility_of_element_located((By.ID,
"finish")))
time.sleep(2)
print("Extracted Text:", hello_text.text)
time.sleep(10)
driver.quit()
```

5. Output & Screenshots: -

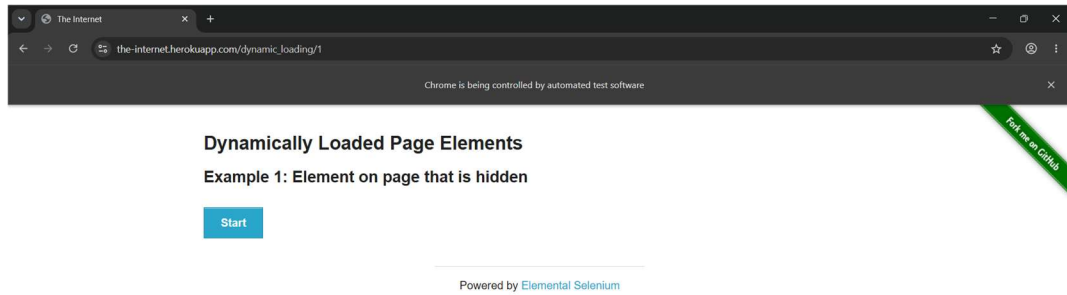


Fig.1: Start button is visible

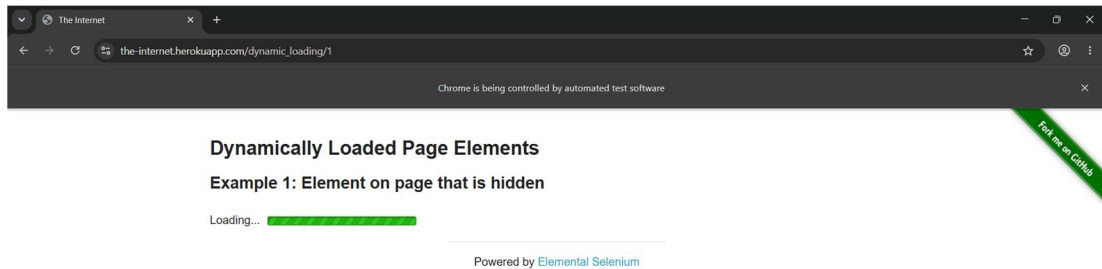


Fig.2: Loading bar is visible

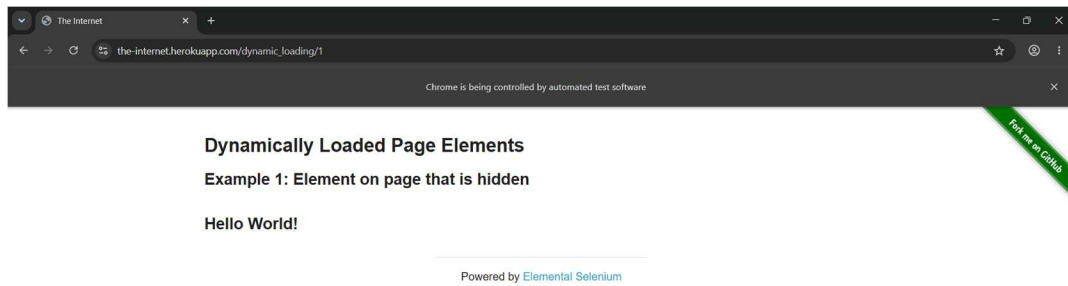


Fig.3: Output is visible

6. Result, Observation, and Conclusion: -

- Result: The script waited precisely for the element to appear without using hardcoded sleeps.
- Observation: Explicit waits poll the DOM until the condition is met, significantly improving test speed and reliability over static delays.
- Conclusion: WebDriverWait is essential for handling asynchronous content effectively.